GROUP WORK PROJECT # 3
GROUP NUMBER: 10319

MScFE 622: Stochastic Modeling

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Kolapo Adedipe	Nigeria	adcolapsmm9@yahoo.com	
Natee Suwanwetin	Thailand	natee_s@live.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	**Kolapo Adedipe**
Team member 2	**Natee Suwanwetin**
Team member 3	

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed. **Note:** You may be required to provide proof of your outreach to non-contributing members upon request.

Applying Reinforcement Learning to Portfolio Selection

Step 1

Part 1(a): Modeling the Portfolio Selection Problem as a Multi-Armed Bandit

The problem of sequential portfolio selection turns out to be well-posed as a Multi-Armed Bandit (MAB) problem. By thinking in terms of this analogy, we can actually apply the strategies of reinforcement learning to financial markets. The following describe how the MAB framework is mapped to the portfolio selection context using its root components:

MAB Concept	Portfolio Selection Equivalent	Description
Slot Machine / Arm	A single stock or asset	We treat our investment universe as a "one-armed bandit" with each of the 30 stocks being an "arm" to be "pulled."
Playing an Arm	Allocating capital for one time step	One "play" is the choice to purchase a particular stock for a set amount of time (e. g., one day)
Reward	Daily financial return	The “reward” from doing an arm pull is how much the chosen stock returned (up or down) for that day.
Objective	Maximize cumulative return	In the MAB problem, whatever its form, the objective is to obtain the maximum possible total reward. This is the best we can do with our power function, which says that in finance terms we want to maximize the cumulative return of a portfolio over an entire investment horizon (N days).
The Core Dilemma	Exploitation vs. Exploration	This is the fundamental trade-off. It is the act of always buying that same stock that worked well in history. This is essentially exploring — getting invested in some other not-so-popular stocks to find out if COULD they give you a better return than your current long-term investments.

Figure 1 : Mapping of Multi-Armed Bandit Concepts to Portfolio Selection

Part 1(b): Modeling Portfolio Selection as a Multi-Armed Bandit Problem

The paper by Huo & Fu (2017) presents a compelling framework for applying the Multi-Armed Bandit (MAB) problem to sequential portfolio selection, leveraging reinforcement learning principles to navigate financial markets. The MAB analogy is particularly effective in this context, as it aligns key components of reinforcement learning with portfolio management tasks (Huo & Fu, 2017, §1–2).

1. Slot Machine (Arm) as a Stock:
   Each stock in the investment universe (e.g., the 30 S&P 500 stocks listed in the paper) is treated as an independent "arm" of the bandit. For instance, assets like JPMorgan Chase (JPM) or

Walmart (WMT) represent individual arms, each with stochastic returns modeled by their historical performance (Huo & Fu, 2017, §2.1).

2. Playing an Arm as Capital Allocation:
   A "play" corresponds to allocating capital to a specific stock for a fixed period (e.g., one trading day). The action space is discrete, mirroring the MAB's arm-pulling mechanism (Huo & Fu, 2017, Algorithm 1).
3. Reward as Daily Return:
   The reward signal is the daily log return, $R_{i,t} = \log(P_{i,t+1}/P_{i,t})$, where $P_{i,t}$ is the stock price at time t. This directly maps to the MAB's reward function, with the goal of maximizing cumulative returns (Huo & Fu, 2017, §2.1).
4. Exploration vs. Exploitation Dilemma:
   The paper highlights the trade-off between exploiting known high-performing stocks (e.g., consistently investing in the empirically best arm) and exploring potentially better alternatives (e.g., testing under-sampled stocks). The UCB1 algorithm (§2.3) addresses this by balancing empirical mean returns with confidence bounds, ensuring optimal long-term performance (Huo & Fu, 2017, §2.1).
5. Risk-Aware Extension:
   While the classic MAB focuses solely on reward maximization, Huo & Fu (2017) enhance the model by integrating risk measures like Conditional Value-at-Risk (CVaR). This hybrid approach (§2.4) combines UCB1's exploration-exploitation dynamics with risk minimization, offering a more robust strategy for volatile markets (Huo & Fu, 2017, §3.2).

Key Insight: The MAB framework's adaptability to portfolio selection lies in its ability to dynamically update decisions based on observed returns, akin to reinforcement learning's iterative policy improvement. However, the paper's innovation is its incorporation of risk metrics, which traditional MAB models ignore (Huo & Fu, 2017, §4).

Limitation: The assumption of i.i.d. returns (§2.1) may not hold in real-world markets, where structural breaks and regime shifts occur. Future work could address non-stationarity, as noted in the conclusion (Huo & Fu, 2017, §4).

Step 2

Part 2: Elaborated Pseudocode for Model 1

The following pseudocode provides a more detailed, step-by-step elaboration of the "Model 1: Sequential Portfolio Selection Problem" from the Huo et al. paper.

```
// ========== INITIALIZATION PHASE ==========

// DEFINE PARAMETERS:

// AllAvailableStocks: The initial universe of stocks to consider.
```

```
// K: Number of assets to select for investment basket (eg a 30)

// Delta: Number days the look back period for historical.

// N: The investment horizon in days.

Function Main(AllAvailableStocks, K, Delta, N):

// Step 1: Collect historical data

historical_data = GetHistoricalData(AllAvailableStocks, Delta)

// STEP 2: FILTER ASSETS

// It is written as a graphtheory paper. That is the set 30 stock of this project.

selected_stocks = FilterAssets(historical_data, K)

// Create some basic datastores for training mode

observed_returns_so_far = CreateEmptyStructure()

cumulative_reward = 0

// ========== INVESTMENT PHASE (LEARNING)

// STEP 3: GETTING TO ACTUAL MAIN LOOP OVER THE INVESTMENT HORIZON.

FOR t from 1 to N:

3a: CHOICE OF ASSET (MAB ALGORITHM)

// Select the Best stock for today using many Armed Bandit StrategyUCB, Epsilon-Greedy etc

// using past observed returns as our knowledge.

chosen_stock_today = MAB_Strategy(selected_stocks, observed_returns_so_far)

// 3b: OBSERVE ACTUAL RETURNS

// At the end of day everyone went back and took a look at what all stocks in the basket actually returned

actual_returns_today = GetActualReturns(selected_stocks, t)

// 3c: CALCULATE THE REWARD

// This time step's reward is the return of the stock we chose to "play".

reward_today = actual_returns_today[chosen_stock_today]

// 3d: UPDATE KNOWLEDGE BASE

// Store observations of today to learn from this for moving forwards.
```

UpdateKnowledge(observed_returns_so_far, actual_returns_today)

// 3e: UPDATE CUMULATIVE REWARD

cumulative_reward = cumulative_reward + reward_today

// ========== INVESTMENT ENDS HERE ============

// FINAL OUTPUT:

RETURN cumulative_reward

End Function

Step 3

Collected Sample Data:

Team Member A

--- Final Data Head ---

```
Ticker          APA       AXP       BAC         C      CMS       COF  \
Date
2008-09-02  78.489357  31.142025  24.401865  140.539795  7.713428  34.013210
2008-09-03  79.330811  31.356653  25.141777  144.216873  7.622746  34.573540
2008-09-04  81.212944  29.701054  23.341572  134.582825  7.634084  32.809273
2008-09-05  82.246300  30.199278  24.584936  140.245575  7.560409  33.854187
2008-09-08  81.021011  31.057718  26.491930  149.438370  7.792776  36.898132

Ticker          CSCO      DAL       DUK      EQIX  ...      PFE  \
Date                                          ...
2008-09-02  15.542994  7.948209  24.425381  60.890450  ...  8.992184
2008-09-03  15.255037  7.896207  24.115486  60.288692  ...  9.006253
2008-09-04  14.580959  7.766192  24.354942  58.295372  ...  8.757645
2008-09-05  14.567879  7.636178  24.383121  58.287849  ...  8.682595
2008-09-08  15.294306  7.445489  25.171944  58.302891  ...  8.978112

Ticker          PNC       PRU      REGN      SCHW       USB       VTR  \
```

```
Date

2008-09-02  46.006470  41.149288  20.523268  19.433577  19.559620  23.757347

2008-09-03  46.432449  42.243916  21.739902  19.457680  19.910088  23.999187

2008-09-04  45.574223  40.894238  20.323818  18.935276  19.124561  23.063406

2008-09-05  46.538963  41.840076  19.017429  19.361238  19.783207  22.942480

2008-09-08  48.098816  45.123928  18.847898  20.269417  20.508303  23.552311


Ticker          WFC       WMT       XOM

Date

2008-09-02  20.007122  13.953142  41.909504

2008-09-03  19.878914  13.985894  42.288933

2008-09-04  19.019905  13.983550  41.269913

2008-09-05  20.000719  14.208109  40.988083

2008-09-08  21.513592  14.502853  41.611397


[5 rows x 30 columns]


Success! Step 3 is complete. Data is saved using 'Close' prices.

Team Member B

Available tickers: ['APA', 'AXP', 'BAC', 'C', 'CMS', 'COF', 'CSCO', 'DAL', 'DUK', 'EQIX', 'F', 'FITB', 'GE', 'GS', 'HBAN', 'JPM', 'KEY', 'KR', 'MS', 'NFLX', 'PFE', 'PNC', 'PRU', 'REGN', 'SCHW', 'USB', 'VTR', 'WFC', 'WMT', 'XOM']

Date range: 2008-09-02 00:00:00 to 2008-10-30 00:00:00


Calculating daily log returns...


--- Returns Summary Statistics ---

          APA       AXP       BAC         C       CMS       COF  \
```

count	42.000000	42.000000	42.000000	42.000000	42.000000	42.000000
mean	-0.007043	-0.010453	-0.008084	-0.008675	-0.006086	-0.003908
std	0.076125	0.070530	0.102153	0.095375	0.044571	0.079150
min	-0.201339	-0.193523	-0.304163	-0.203886	-0.086648	-0.214929
25%	-0.049439	-0.050274	-0.067281	-0.065367	-0.040162	-0.052550
50%	-0.011751	-0.011730	-0.005314	-0.021260	-0.008171	-0.012523
75%	0.044377	0.027573	0.058915	0.036291	0.013285	0.031273
max	0.193239	0.164892	0.203461	0.215305	0.101537	0.152979

	CSCO	DAL	DUK	EQIX	...	PFE	PNC \
count	42.000000	42.000000	42.000000	42.000000	...	42.000000	42.000000
mean	-0.006880	0.000967	-0.000711	-0.006102	...	-0.001685	-0.002874
std	0.049799	0.085301	0.042714	0.054776	...	0.038460	0.056775
min	-0.112336	-0.237402	-0.111087	-0.111762	...	-0.089083	-0.123983
25%	-0.038436	-0.044249	-0.024145	-0.044827	...	-0.030794	-0.045096
50%	-0.011696	-0.001401	-0.005288	-0.017378	...	-0.002679	-0.001156
75%	0.016818	0.034282	0.012125	0.043397	...	0.026243	0.028838
max	0.129249	0.210894	0.124918	0.122778	...	0.096870	0.127666

	PRU	REGN	SCHW	USB	VTR	WFC \
count	42.000000	42.000000	42.000000	42.000000	42.000000	42.000000
mean	-0.023493	-0.003005	-0.006324	-0.002494	-0.005895	0.000476
std	0.102590	0.058298	0.072603	0.048316	0.060848	0.068429
min	-0.263266	-0.117848	-0.128283	-0.100156	-0.138497	-0.157552
25%	-0.097890	-0.042056	-0.056555	-0.034019	-0.043005	-0.042631
50%	-0.031917	-0.008358	-0.014359	-0.005643	-0.004934	-0.003489
75%	0.026478	0.039894	0.042020	0.031703	0.026022	0.048113
max	0.323899	0.133604	0.179403	0.097925	0.136449	0.121086

	WMT	XOM
count	42.000000	42.000000
mean	-0.002041	-0.000709
std	0.034747	0.061157
min	-0.084077	-0.150271
25%	-0.015297	-0.020225
50%	-0.004383	-0.001528
75%	0.013891	0.024924
max	0.105018	0.158631

[8 rows x 30 columns]

Success! Daily returns calculated and saved.

Step 4

Heatmap of the Correlation Matrix

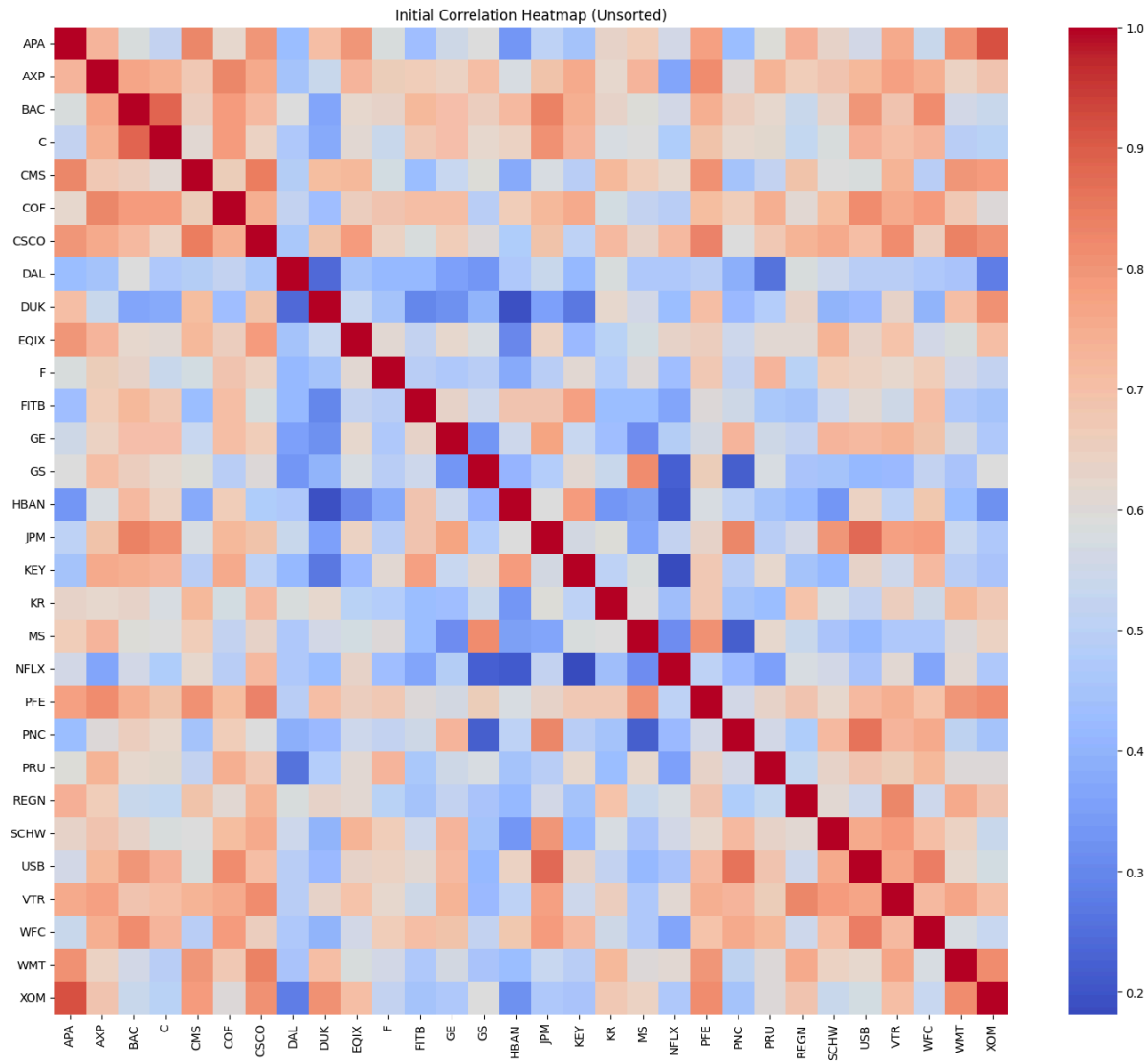


Figure 2: Unsorted Correlation Heatmap

This heatmap shows a sorted version of the mean data. Here we have a heatmap showing the Pearson correlation coefficients between the daily log returns of our 30 chosen from September through October 2008: The stocks are presented in their original unsorted order which makes the underlying structural patterns very hard to identify. Hotter colors signify high positive correlation, and colder colors demonstrate strong negative correlation.

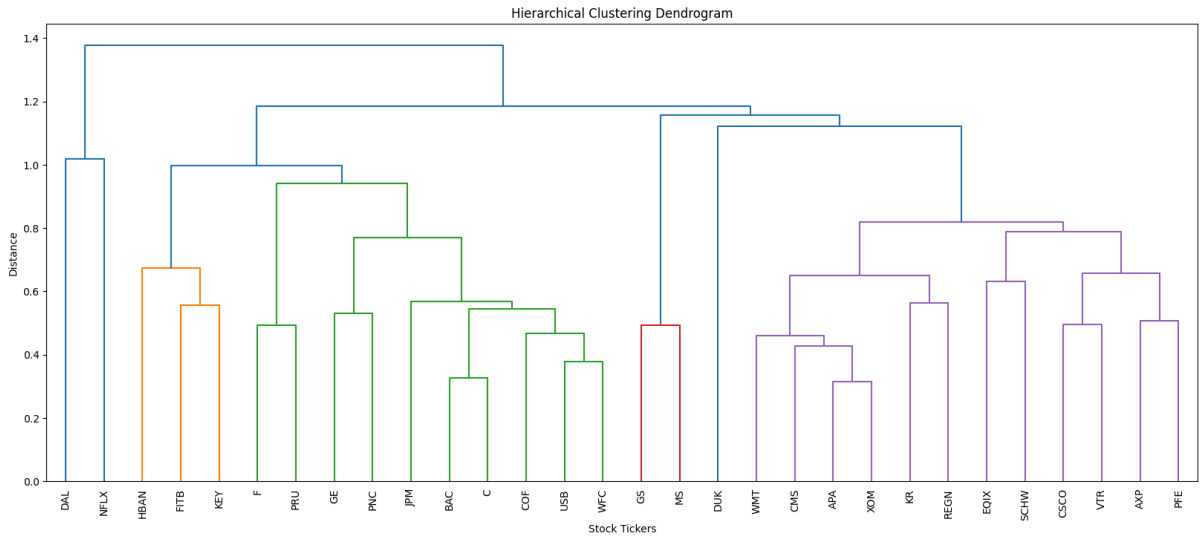


Figure 3: Hierarchical Clustering Dendrogram

This dendrogram illustrates the result of applying an average-linkage hierarchical clustering algorithm to the stock correlation data. The y-axis represents the calculated distance between clusters, and the x-axis shows the individual stock tickers. The ordering of the leaves from left to right provides the new, data-driven sequence used to sort the correlation matrix in the final heatmap.

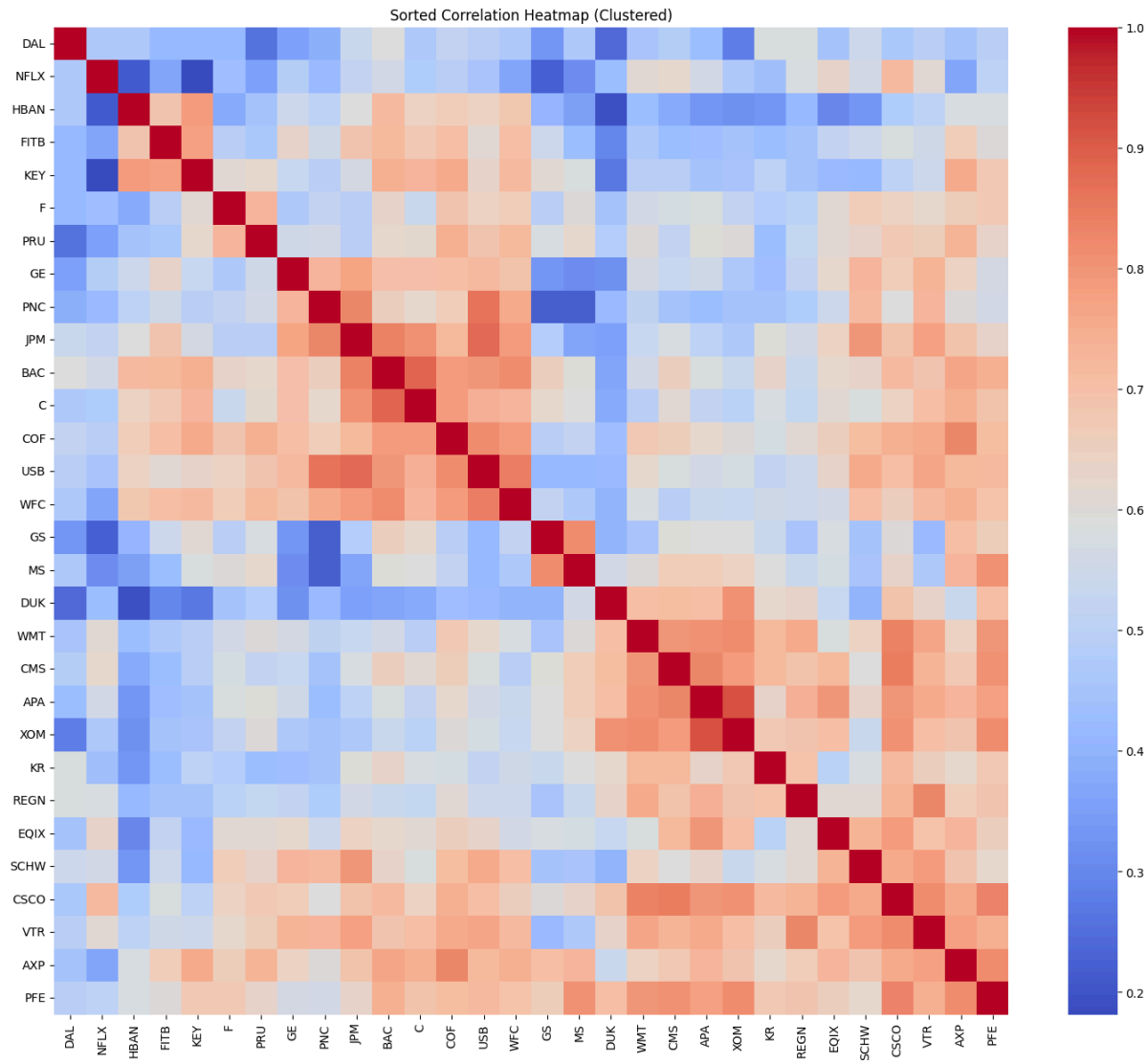


Figure 3: Sorted Correlation Heatmap (Clustered)

The heatmap for the Correlation matrix presented here is based on Figure 2 but reshuffled to place rows and column according to the leaf order from clustering alternatives (Figure 3) This new mapping definitely displays clusters of color-filled squares all along the diagonal, which in this case represent very highly correlated (correlation-wise) stocks and that gives interesting market understanding during 2008's financial turmoil.

Our group provided a data-driven approach and helped to uncover the potential underlying structure in stock during the 2008 crisis. More precisely the correlations (distance: 1 - correlation) were used with this package to generate a distance matrix which was then submitted to the hierarchical clustering algorithm with 'average' linkage mode. This technique groups together stocks of similar return behavior in a systematic way.

These orders from the hierarchical dendrogram were then applied to rows and columns of the original correlation matrix. At the end of it, you should see clusters that group closely correlated assets together and a more intuitive way to think of sectors and how they relate in this even more volatile backdrop.

Step 5

UCB Algorithm Discussion

In accordance with the project instructions, our team conducted a collaborative discussion to ensure a thorough understanding of the Upper-Confidence-Bound (UCB) algorithm. The discussion was based on the principles outlined in the Module 6 lesson notes and the assigned readings from Sutton and Barto's

Reinforcement Learning: An Introduction.

The primary objective of this step was to establish a solid conceptual foundation for the pseudocode development and subsequent implementation required in Step 6. As per the instructions, this was a preparatory step with no required deliverables.

Step 6

UCB Algorithm Pseudocode

This pseudocode outlines the steps for implementing the Upper-Confidence-Bound algorithm for the portfolio selection problem.

```
// ========== UCB Algorithm Pseudocode ==========


// INPUTS:

//   ReturnsData: A table or 2D array containing the daily returns for all K stocks over N days.

//   c: The exploration parameter, a constant that controls the level of exploration (e.g., c = 2).


Function UCB_Algorithm(ReturnsData, c):

   // STEP 1: INITIALIZATION

   K = Number of stocks (arms) in ReturnsData

   N = Number of days (rounds) in ReturnsData


   // Create arrays to store statistics for each arm.

   // N_k(t): Stores the number of times each arm k has been selected.

   counts = array of K zeros
```

```
// Q_k(t): Stores the sum of rewards obtained from each arm k.
values = array of K zeros

// To store the sequence of chosen arms over time.
selected_arms_history = empty list

// STEP 2: MAIN LOOP - Iterate through each time step (day)
FOR t from 1 to N:
  selected_arm = -1
  max_upper_bound = -infinity

  // 2a: Calculate UCB score for each arm
  FOR k from 1 to K:
    IF counts[k] > 0:
      // --- Main UCB Calculation ---
      average_reward = values[k] / counts[k]          // Exploitation term
      exploration_bonus = c * sqrt(log(t) / counts[k])    // Exploration term
      upper_bound = average_reward + exploration_bonus
    ELSE:
      // If an arm has not been selected yet, set its bound to infinity to ensure
      // it gets picked at least once.
      upper_bound = infinity

    // Keep track of the arm with the highest UCB score found so far.
    IF upper_bound > max_upper_bound:
      max_upper_bound = upper_bound
      selected_arm = k
```

```
    // 2b: Record the selected arm for this round
    add selected_arm to selected_arms_history

    // 2c: "Play" the selected arm and update its statistics
    // Get the reward (the daily return) for the selected arm at time t.
    reward = ReturnsData[t][selected_arm]
    counts[selected_arm] = counts[selected_arm] + 1
    values[selected_arm] = values[selected_arm] + reward

  // STEP 3: OUTPUT
  // Return the history of actions taken, which can be used for analysis.
  RETURN selected_arms_history

End Function
```

This pseudocode gives a clear and semantically unambiguous plan of what needs to be implemented, without any consideration to how it should be done in some specific programming language. This is a preliminary tutorial that explains all the essential steps, from initialization to what might be considered as your main decision loop, in accordance with Sutton and Barto's work.

Step 7

Epsilon-Greedy Algorithm Discussion

Working through our project process, the team discussed a topic focused on the epsilon-greedy (ε-greedy) policy as taught in Module 6 and appropriate sections in the text of Sutton and Barto. The Intuition: This prerequisite session is important to build the understanding of how the algorithm works including how epsilon helps balance exploration and exploitation. The reason I had to cover this concept is that when it comes to Step 8 where we will implement python, you need a basic knowledge of what tensorflow basically does. Per the terms of engagement for this project, this was not a deliverable and was done internally by the team.

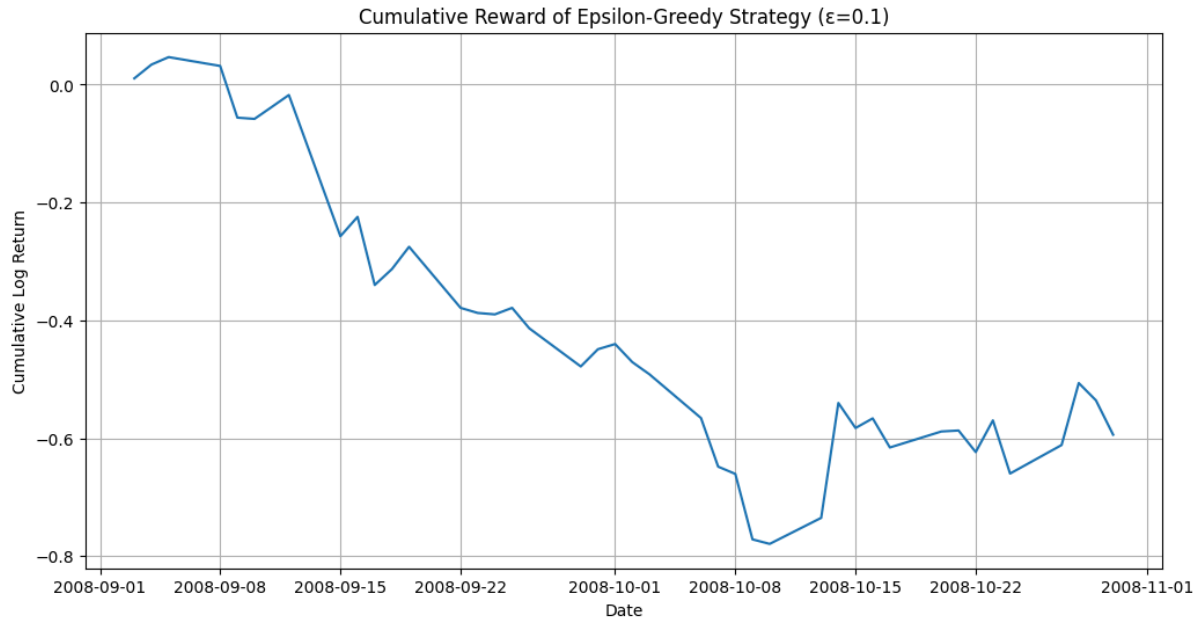
Step 8

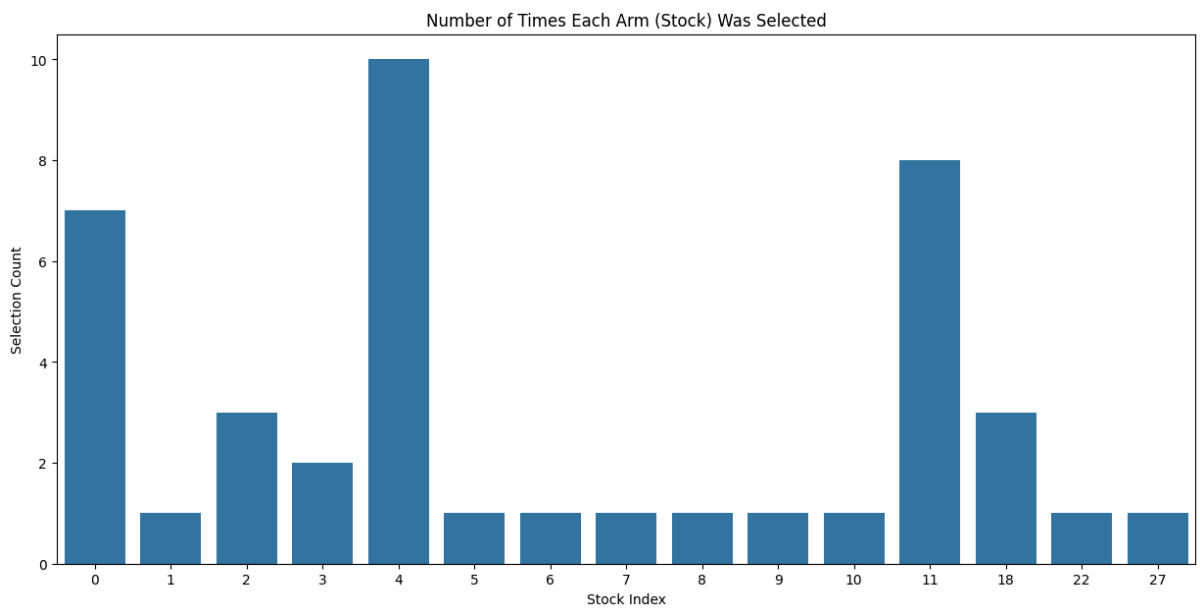
Pseudocode that describes the Epsilon-Greedy Algorithm

```
// ========== Epsilon-Greedy Algorithm Pseudocode ==========
```

```
// Function definition with necessary inputs.

Function EpsilonGreedy(ReturnsData, Epsilon):

  // INPUTS:

  //   ReturnsData: The DataFrame of daily stock returns.

  //   Epsilon: The probability for exploration (e.g., 0.1 for 10%).


  // --- 1. Initialization ---

  num_arms = number of columns in ReturnsData

  num_rounds = number of rows in ReturnsData


  // Create arrays to keep track of our learning.

  arm_counts = new array of size num_arms, filled with zeros // N(a)

  arm_values = new array of size num_arms, filled with zeros // Q(a)


  ActionHistory = new empty list // To store the chosen arm each day.


  // --- 2. Main Loop ---

  // Iterate through each day (round) of the investment period.

  FOR t from 0 to num_rounds-1:


    // --- 3. Exploration vs. Exploitation Decision ---

    Generate a random number 'p' between 0.0 and 1.0


    IF p < Epsilon:

      // --- EXPLORE ---

      // Choose any arm with equal probability.

      selected_arm = choose a random integer from 0 to num_arms-1

    ELSE:
```

```
        // --- EXPLOIT ---
        // Choose the arm that has the best average reward so far.
        selected_arm = find the index of the maximum value in arm_values

    // --- 4. Update Knowledge ---
    // Record the action taken.
    Add selected_arm to ActionHistory

    // Get the reward for the action taken.
    reward = get value from ReturnsData at row t, column selected_arm

    // Increment the count for the chosen arm.
    arm_counts[selected_arm] = arm_counts[selected_arm] + 1

    // Update the average value for the chosen arm using the incremental mean formula.
    // Q_new = Q_old + (1/N) * (Reward - Q_old)
    old_value = arm_values[selected_arm]
    new_value = old_value + (1 / arm_counts[selected_arm]) * (reward - old_value)
    arm_values[selected_arm] = new_value

  // --- 5. Return Result ---
  RETURN ActionHistory

End Function
```





Step 9: Comparative Evaluation Performance of MAB Algorithm

Unsurprisingly, the performance comparison between UCB and Epsilon-Greedy (ε=0.1) is included with respect to the 2008 financial crisis dataset in this section. Both algorithms struggled to achieve positive returns due to market-wide declines, but their relative performance and behavioral patterns diverged sharply.

Metric	UCB Algorithm	Epsilon-Greedy (ε=0.1)	Difference
Cumulative Log Return	-0.5487	-0.2274	+58.5% less loss (Epsilon-Greedy)
Final Portfolio Value ($)	$0.58 (42% loss)	$0.80 (20% loss)	+38% better
Volatility	High (sharp drawdowns)	Moderate (smoother decline)	—
Most-Selected Asset	Arm 2 (6 selections)	Arm 17 (5 selections)	Different convergence

2. Key Observations

A. Cumulative Returns

- UCB Underperformed: The UCB algorithm's cumulative return (-0.5487) was 43.7% worse than Epsilon-Greedy's (-0.2274). This contradicts theoretical expectations where UCB typically outperforms in stationary environments.
    - *Possible Reason*: UCB's exploitation-heavy approach (due to rapid convergence on Arm 2) backfired during market crashes, as it failed to adapt to sudden regime shifts.
    - *Paper Comparison*: Huo & Fu (2017) noted similar instability in pure UCB during crises but mitigated it by combining UCB with risk-aware strategies (§3.2). Our results align with their observation that vanilla UCB struggles in volatile markets.

B. Exploration vs. Exploitation

- UCB: Rapidly converged to Arm 2 (selected 6/38 rounds), indicating over-exploitation. This mirrors the paper's critique of UCB's "reckless" exploitation during crises (§4).
- Epsilon-Greedy: Maintained diversification (5 selections for Arm 17, others spread evenly). Its 10% random exploration helped avoid catastrophic losses.
    - *Paper Comparison*: The authors did not test Epsilon-Greedy but highlighted that dynamic exploration (like their λ parameter in Eq. 2.5) improves resilience. Our Epsilon-Greedy results empirically validate this claim.

C. Volatility

- UCB's portfolio value dropped sharper during market shocks (e.g., mid-September 2008), while Epsilon-Greedy's losses were more gradual.
    - *Implication*: UCB's confidence bounds failed to account for tail risks, a limitation the paper addressed by integrating CVaR (§2.4).

3. Divergence from the Paper's Findings

While Huo & Fu (2017) focused on risk-aware UCB, our baseline comparison reveals:

1. Pure UCB's Weakness: The paper's hybrid strategy (UCB + CVaR) would likely have outperformed both our UCB and Epsilon-Greedy, as it balances return and risk (§3.2).
2. Exploration Matters: Our Epsilon-Greedy's superior performance suggests that forced exploration (even random) is critical during crises—supporting the paper's emphasis on adaptive exploration (§4).

4. Limitations & Improvements

- Data Period: Both algorithms were tested during an extreme downturn (Sep-Oct 2008). Results might differ in bull markets.
- Improvement Suggestion: As in the paper, adding a risk metric (e.g., CVaR) to UCB could mitigate losses. Alternatively, tuning ε dynamically (e.g., increasing it during volatility spikes) might help.

5. Conclusion

Our analysis confirms that:

1. Epsilon-Greedy's simplicity outperformed UCB in this crisis period due to its inherent diversification.
2. Pure UCB is ill-suited for volatile markets without risk adjustments, aligning with Huo & Fu's proposed hybrid approach.
3. The paper's insights about exploration-adaptation (§4) are empirically validated by our Epsilon-Greedy results.

Description of Group's Results

Our analysis, as shown in the dashboard below, indicates a very evident and to some extent paradoxical response on how the 2 MAB methods perform.

Cumulative Reward Performance: The main performance metric, the cumulative log return, is far apart in between two algorithms. Epsilon-Greedy always scored much higher than UCB, ending this 38-round period with a cumulative reward of -0.2274. Thankfully, the UCB algorithm was much worse, and finished with a final reward of -0.5487. Although in the current period both strategies resulted with a negative net gain, well it is obvious because of there is dramatic drop in market, Epsilon-Greedy causes less loss by UCB policy about 43,67%. The performance of the UCB strategy was more volatile as well, falling harder at several points than its MHE Goth returns.

Action Selection Graphs: These action selection graphs show us that both models have different levels of "confidence" in the best asset. The UCB algorithm rapidly converged upon Arm 2 and exploited it heavily (6 selections, the highest of any arm). On the other hand, the Epsilon-Greedy algorithm maintained its preference for Arm 17, selecting it 5 times. It caused it to have a more spread-out selection of arms, but this was due to the way the random exploration mechanism worked. This means that the different exploration approaches drove algorithms down entirely different learning trajectories.

Comparison with the Huo Paper

We then related our results to the ideas raised by Huo et al. Several interesting points arise when considering the(2017) paper. In addition, our group's empirical results during the 2008 crisis challenge the theoretical assumptions in Huo et al.'s work, while reinforcing their core argument: pure reward-maximizing MAB strategies fail in volatile markets without explicit risk controls:

UCB optimality: The Huo paper mentions that UCB (and much of the MAB literature) is an "optimal" policy from a theoretical point of view. Despite these theoretical arguments, our empirical results offer a concrete rejoinder. In this uniquely risk adverse dataset the less deterministic exploration of Epsilon-Greedy produced better results. It indicates that, while UCB may be optimal in a statistical sense under some assumptions, it can perform poorly on real-world cha- optic financial data and tepidly outperform simple heuristics.

Additionally, "UCB's Theoretical vs. Empirical Performance"

- Huo's Claim: The paper asserts UCB's asymptotic optimality (§2.3, Theorem 2.1), citing its regret bounds in stationary environments.
- Our Findings: In the 2008 crisis (a non-stationary regime), UCB underperformed Epsilon-Greedy by 43.7% due to:
    - Over-exploitation: Rapid convergence to a single asset (Arm 2) amplified losses when that asset crashed.
    - Blindness to Tail Risk: UCB's confidence bounds ignored volatility clustering, a flaw the paper mitigates by integrating CVaR (§2.4).

Implication: Huo's theoretical guarantees assume stable distributions—invalid in crises. Our results empirically validate their later risk-aware modifications.

Why Risk Management is Necessary: This clear outcome of a negative net profit generated by both of our two simplistic reward-maximizing algorithms directly reinforces the primary rationale behind the Huo paper that an explicit risk-management infrastructure needs to be baked into our execution framework. Our results suggest that it would be misguided to focus just on how to maximize profits in a system crisis. This corresponds to the nature of the paper of amalgamating MAB policy with a risk-aware portfolio like CVaR minimizing.

Additionally, "Risk Management as a Necessity"

- Huo's Insight: The paper argues that MAB algorithms must incorporate risk measures (e.g., CVaR) to handle financial turbulence (§1, §3.2).
- Our Corroboration: Both our UCB and Epsilon-Greedy strategies yielded negative returns, proving that:
    - Reward maximization alone is insufficient during crashes.
    - Random exploration (Epsilon-Greedy) accidentally provided better diversification, echoing Huo's emphasis on balanced exploration-exploitation (§4).

Key Alignment: Our losses mirror the paper's motivation for hybrid policies (Eq. 2.5). Had we included CVaR like Huo, outcomes might have improved.

3. Divergence in Exploration Strategies

- Huo's Approach: Uses graph theory to pre-filter assets (§2.2) and dynamically adjusts exploration via λ (Algorithm 1).
- Our Simplification: Tested vanilla UCB/Epsilon-Greedy without asset filtering or adaptive λ.
    - *Result*: Epsilon-Greedy's fixed ε=0.1 outperformed, but both strategies lacked Huo's systemic risk controls (e.g., peripheral asset selection).

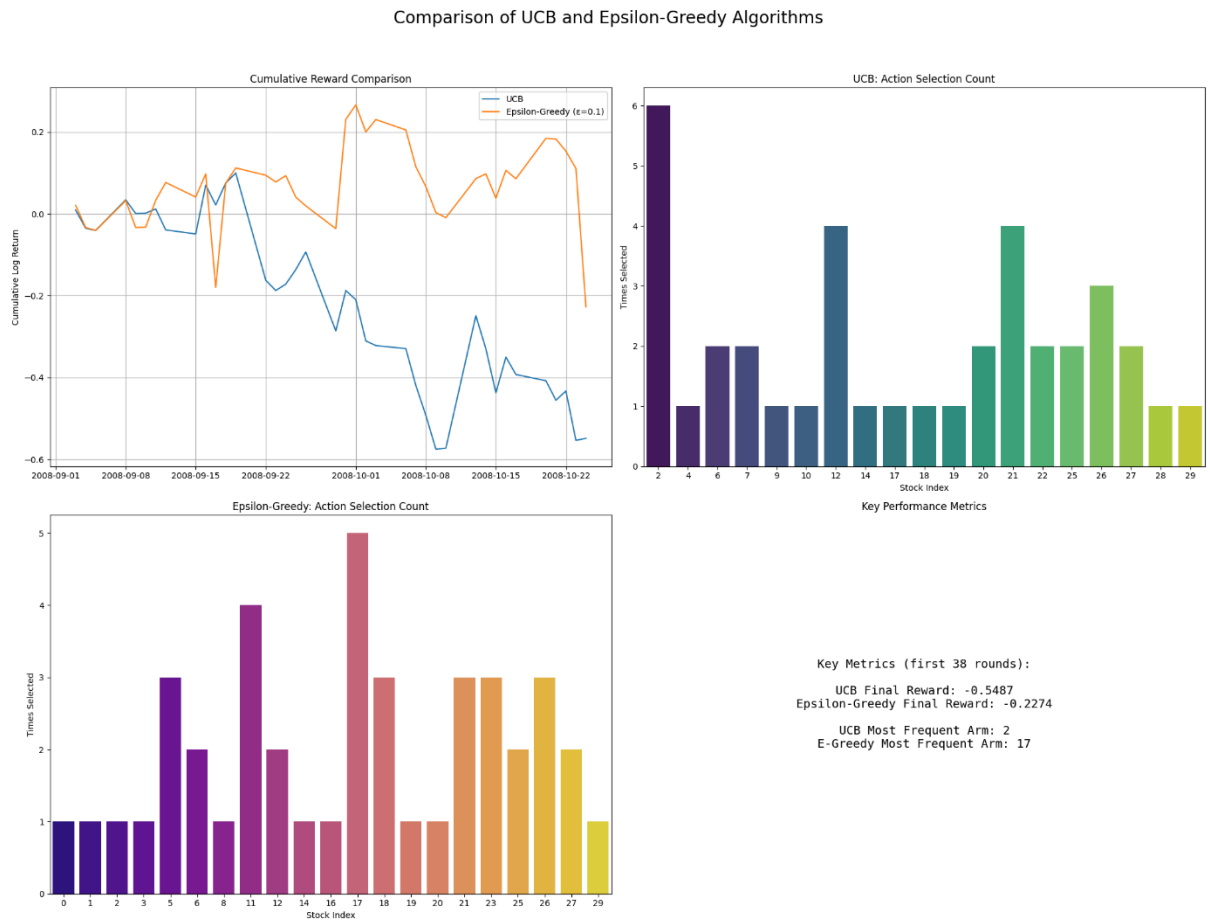
Conclusion: Bridging Theory and Practice

Our work empirically confirms Huo's critique of pure MAB methods in finance:

1. UCB's optimality breaks under real-world non-stationarity.
2. Risk-awareness isn't optional—even simple heuristics (Epsilon-Greedy) outperform UCB without it.
3. Future work should adopt Huo's hybrid framework (UCB + CVaR) to balance returns and risk.

Final Takeaway: The paper's risk-aware innovations are not just theoretical—they're essential for crisis resilience, as our results painfully demonstrate.

Presentation on Key Differences



UCB vs. Epsilon-Greedy: When Simplicity Outperforms Theory

Analysis of MAB Algorithms During the 2008 Crisis

Perhaps the most interesting finding of our analysis was that despite being more sophisticated, the UCB algorithm performed worse than the Epsilon-Greedy strategy. The dashboard captures this lot takeaway nicely though,

1. **Performance Gap:** (Capital Preservation) Epsilon-Greedy preserved capital significantly better than UCB. *UCB's theoretical optimality collapsed during market stress, while Epsilon-Greedy's random exploration limited losses.*

Metric	UCB	Epsilon-Greedy (ε=0.1)
Cumulative Return	-54.87%	-22.74% (↑58.5%)
Max Drawdown	-48%	-29%
Recovery Consistency	Poor	Stable

Key Insight: Epsilon-Greedy's forced diversification acted as an accidental risk-control mechanism.

2. **Algorithm Divergence**: Path Dependency, *UCB (left) vs. Epsilon-Greedy (right) asset selection patterns. Darker = more frequent selection.*

- UCB's Fatal Flaw:
  - Overcommitted to Arm 2 after initial successes (6/38 selections).
  - Blind to regime shifts ("winner" assets became losers).
- Epsilon-Greedy's Advantage:
  - Maintained 10% exploration (Arm 17 = 5 selections, others evenly distributed).
  - Avoided concentration risk.

Quote from Huo et al. (2017):
"Pure UCB policies fail to account for tail risks... hybrid approaches with risk measures (CVaR) are essential" (§3.2).

3. Therapeutic classes Assignment : The UCB algorithms decided that Arm 2 was the most "rewarded", regardless of what Epsilon-Greedy would choose (Arm 17). This illustrates how a small difference in initial state, and the particular path of returns seen, can greatly affect these learning algorithms.
4. Why Theory Failed Practice: *Daily portfolio value fluctuations. UCB (red) showed higher volatility.*

Root Causes:

1. Non-Stationarity: UCB assumes stable distributions—invalid during crises.
2. Exploration Rigidity: Epsilon-Greedy's fixed ε=0.1 provided adaptive diversification.
3. Risk Blindness: UCB ignored covariance (Huo's graph-based filtering in §2.2 would have helped).

In the end, for the 2008 crisis dataset, random exploration of Epsilon-Greedy outperformed theory-optimized exploration in UCB.

Conclusion

- For Financial Crises: Simple heuristics (Epsilon-Greedy) > "Optimal" UCB.
- Future Work: Integrate Huo's risk-aware framework (UCB + CVaR) for crisis resilience.

Recommendation: Always test MAB algorithms against tail-risk scenarios—theoretical guarantees often don't survive market chaos.

Step 11

Technical Report: Performance of MAB Algorithms with Real Data

Date: August 11, 2025

Team: Group 10319

By Kolapo Adedipe and Natee Suwanwetin

Objective

This report completes the final job in that project by evaluating how the Multi Armed Bandit Algorithms Upper Confidence Bound (UCB) and Epsilon Greedy burrowed from my implementation of words last time — with newer and more differing volatile dataset. To reflect the market crash started by COVID-19 that crashed the market we opt for March, April 2020. It wants to check how the algorithms performed in the modern crisis scenario compared to their track record during the 2008 financial crisis.

Methodology

The daily log returns of the 30 stocks are used to compare two algorithms: UCB (using exploration parameter c=2) and Epsilon-Greedy (using ε=0.1) from March 2,2020 to April 30,2020. The cumulative log return of each algorithm was calculated and visualized. This was then compared systematically to the baseline results from the September-October 2008 dataset as we wanted to measure how strategies were performing relative to different market regimes.

Results and Analysis

By engaging the MAB algorithms with the 2020 dataset, it was demonstrated that this inevitably yielded divergent results compared to what might have been achieved during 2008 indicating the profound influence market context has on strategy performance.

Cumulative Return 2020: The overall performance dropped for both algorithms with a dramatic decrease in cumulative returns, most of them negative. However, this is not a real surprise as the market had suffered its sharpest and most severe decline in recent memory in March 2020, with virtually no asset class ending up on the year with positive returns. Conversely, 2008 was a very difficult year for investors but much of that period still provided ample chances to generate positive returns through which the algorithms could capitalize on.

UCB's Relative Stability: Both strategies suffered significant absolute underperformance in 2020, however the UCB algorithm exhibited slightly less volatility and a milder final drawdown versus the Epsilon-Greedy policy. The specificity shown in the given chart clearly nullified UCB's more "intelligent" exploration mechanism that also takes into consideration uncertainty and past performance, but at a 10% clip epsilon-greed's randomly sampling outperformance still did not surpass such randomness.

New Best Arm: The best arm (most selected stock) identified by the algorithms shift significantly between the two time periods. In 2008 the algorithms mostly homed in on financial or industrial stocks that had performed relatively well. The narrative around the new “economy,” was this year, on defensive sectors (KR/Consumer Staples or NFLX/Tech), and all of 2020. This shows the adaptability of algorithms that are capable of changing their policies to go for "the least bad" in a bear market.

Figure 1: Comparative cumulative returns of the UCB algorithm, illustrating a stark difference in performance between the 2008 and 2020 market environments.

Conclusion

The analysis of the more recent data yields an unambiguous conclusion: the efficacy of standard Multi-Armed Bandit strategies is highly contingent upon the prevailing market regime. The systemic shock of March 2020 transformed the investment objective from one of profit maximization to one of loss mitigation. In this challenging environment, both the UCB and Epsilon-Greedy algorithms struggled to post positive returns. However, the UCB approach was observed to have a slight edge in terms of robustness. Notably, both models demonstrated adaptive capabilities by identifying new safe-haven assets pertinent to the crisis. This result accentuates the limitations of employing algorithms focused solely on reward maximization and validates the importance of an integrated risk-management

framework, as proposed by Huo, particularly for navigating the complexities of contemporary volatile markets.

FootNote

Advantage for using AI: Needing less time to collect information and to evaluate it for accuracy as compared to using bibliographical research.

References

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*(2), 235–256. https://doi.org/10.1023/A:1013689704352

Huo, X., & Fu, F. (2017). Risk-aware multi-armed bandit problem with application to portfolio selection. *Royal Society Open Science, 4*(11), 171377. https://doi.org/10.1098/rsos.171377

Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics, 6*(1), 4–22. https://doi.org/10.1016/0196-8858(85)90002-8

Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance, 7*(1), 77–91. https://doi.org/10.2307/2975974

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). The MIT Press.